

StageBoard — Project Status & Technical Documentation

As of commit `3d39837` , 2026-08-26 · 51 commits · branch `main`

StageBoard is a modular, local-first, all-in-one live operating system for bands: it synchronizes setlists, chords, in-ear-monitoring, MIDI cues and lighting control over a local mesh network on stage. It is explicitly inspired by Home Assistant's modular dashboard approach. This document describes the **as-built** state of the codebase — what actually exists and runs today — as distinct from the project's vision documents (`docs/01` – `docs/08`), which describe the long-term target architecture. Every claim below is sourced directly from the code.

1. Core Principles (as realized in code)

The project's three stated principles (from `CLAUDE.md`) are all visible in the current implementation, not just the vision docs:

- 1. Local-First & Resilience.** Every workspace-scoped piece of data (songs, variants, setlists, show state, plugins, profiles, dashboards, show log) lives in a per-workspace **PouchDB** database in the browser, replicated live to a CouchDB instance when one is configured (`VITE_COUCHDB_URL`). If no CouchDB URL is set, the app still runs fully offline on local PouchDB alone. The Stage-Server (`core-backend`) is a thin Fastify layer added on top for real-time show-control triggers and outbound network calls (tab scraping, MusicBrainz) that a browser can't safely do itself — it is not the source of truth for the data model.
 - 2. Graceful Degradation.** Every widget that depends on external hardware/plugins is wrapped in `WidgetFrame` , which greys the widget to 50% opacity and overlays an "Offline / Hardware nicht erreichbar" badge when its capability is `degraded` — the widget stays in its exact grid position rather than disappearing, preserving a musician's muscle memory. A widget whose plugin isn't installed at all (`missing`) simply never appears in the widget picker to begin with.
 - 3. Progressive Disclosure.** The live dashboard is read-only by default; a 600ms long-press on a lock icon (`EditLock.tsx`) is required to enter Edit Mode, and Edit Mode is deliberately **never persisted** — every reload comes back locked, so a forgotten edit session can't be misused live on stage.
-

2. Architecture Overview

Monorepo (npm workspaces, `packages/*`):

Package	Purpose	Stack
<code>packages/shared-types</code>	Zod schemas + TypeScript interfaces shared by frontend and backend	Zod 4, TypeScript 7
<code>packages/core-backend</code>	Fastify HTTP server: show-control plugin gateway, lookup/scrapper plugins, CouchDB plugin-sync	Fastify 5, <code>puppeteer-core</code> , <code>tsx</code> , Vitest
<code>packages/stage-pwa</code>	The tablet/PC client PWA: dashboard/widget system, song editor, live show UI	React 18, Vite 8, Zustand 5, PouchDB (<code>pouchdb-browser</code>), <code>react-grid-layout</code> , Tailwind CSS

Node version is pinned via `nvm / .nvmrc` to Node 24 (not the system `apt` package, which on this Ubuntu install only offers an unpatched Node 18).

Data flow, in one sentence: every tablet keeps its own local PouchDB per workspace; `db.sync(remoteDb, {live:true, retry:true})` continuously replicates it against a per-workspace CouchDB database; the Stage-Server watches the same CouchDB databases (via long-polling `_changes`) to learn which plugins are installed and to run scraping/lookup requests on the network's behalf; there is no other transport (no WebSocket/SSE) anywhere in the system.

Infrastructure: `docker-compose.yml` runs a single `couchdb:3` container (admin/admin, dev-only credentials, persisted to a named volume). `scripts/setup-couchdb.sh` is a re-runnable bootstrap script: waits for CouchDB to become reachable, finishes single-node cluster setup (`_cluster_setup`), which provisions `_users / _replicator / _global_changes`), creates the app database, and enables CORS (`httpd/enable_cors`, `cors/origins`, `cors/credentials`, `cors/methods`, `cors/headers`) for browser-side PouchDB sync. No per-band/per-tablet CouchDB user accounts are provisioned — everything authenticates as the single admin user baked into the compose file.

3. Data Model (`packages/shared-types`)

All schemas are Zod v4 objects, re-exported flat from `index.ts`. Every workspace-replicated document type is listed below with every field.

3.1 Song (`song.ts`)

A `Song` is deliberately thin — it is the stable identity of a piece of music, not its playable content (that lives in variants, see 3.2).

Field	Type	Notes
<code>id</code>	<code>string</code> (min 1)	required
<code>title</code>	<code>string</code> (min 1)	required
<code>bpm</code>	<code>number</code> (positive)	required — kept as a read-compatibility mirror of the default variant's bpm; every not-yet-migrated consumer still reads this

Field	Type	Notes
chordProContent	string	required — same mirror role
timecodes	TimecodeMarker[]	.default([]) — same mirror role
artist	string	optional — lives on Song , not SongVariant , because the band doesn't change between arrangements of the same song

TimecodeMarker: { timeMs: number (int, ≥0), label: string (min 1) }.

3.2 SongVariant (songVariant.ts)

A fully self-contained, independently playable arrangement of a song (e.g. "Original", "Akustik", "Kurzfassung Firmenfeier"). The schema's own comment is explicit about why this is a **full copy** rather than a delta/override on Song : a nullable "inherit from Song" model would need resolution logic duplicated everywhere bpm/content is read (PrompterWidget, computeQueue , the tuner's reference pitch, ...); a self-contained copy needs none of that, and ChordPro text is cheap enough to duplicate.

Field	Type	Notes
id	string (min 1)	required
songId	string (min 1)	required
label	string (min 1)	required
isDefault	boolean	required, no default — exactly one variant per song should carry this
bpm	number (positive)	required
chordProContent	string	required
timecodes	TimecodeMarker[]	.default([])
tracks	TrackMeta[]	.default([])
key	string	optional — absent (not blank) when unknown, since a blank string would be indistinguishable from "known to have no key"
tuning	string	optional, e.g. "E A D G B E" or "Drop D"
capo	number (int, ≥0)	optional — absent (not 0) means "no capo", matching how Ultimate Guitar's own data only emits this field when a capo is actually used

TrackMeta (one attached audio file):

Field	Type	Notes
id	string (min 1)	also the PouchDB attachment key, namespaced track- <code>{id}</code>
kind	'reference' 'band-mix' 'stem'	reference = learning aid (e.g. an extracted YouTube recording), never the source of truth for a live show; band-mix

Field	Type	Notes
		= the band's own backing track; <code>stem</code> = one isolated instrument derived from another track
<code>label</code>	<code>string</code> (min 1)	e.g. "Referenz", "Band-Playback"
<code>source</code>	<code>'upload' 'youtube-extract' 'stem-separation'</code>	how the track was produced
<code>parentTrackId</code>	<code>string null</code>	which track a stem was derived from; <code>null</code> for anything uploaded/extracted directly
<code>mimeType</code>	<code>string</code>	
<code>addedAt</code>	<code>number</code> (int, ≥0)	epoch ms

3.3 Setlist (`setlist.ts`) — entry-based model

`SetlistEntry`: { `id`: `string`, `songId`: `string`, `variantId`: `string | null` }. The entry's own `id` — distinct from `songId` — is what lets the same song appear twice in one setlist under two different variants (e.g. the full version early in the set, a shortened "Kurzfassung" as the encore); `variantId`: `null` means "play this song's `isDefault` variant."

`Setlist`: { `id`: `string` (min 1), `name`: `string` (min 1), `entries`: `SetlistEntry[]` } (required array, no default). Legacy setlists persisted before this model existed (`songIds`: `string[]` , no `entries`) are migrated lazily and idempotently at read time in `useSetlistsStore`'s `toSetlist()` , synthesizing one entry per song id.

3.4 ShowState (`showState.ts`)

A **singleton**, per-workspace synced document: which setlist/song is active, and which tablet holds write authority.

Field	Type	Notes
<code>activeSetlistId</code>	<code>string null</code>	
<code>activeEntryId</code>	<code>string null</code>	the current <i>entry</i> 's id, not a bare <code>songId</code> — needed to disambiguate two occurrences of the same song. With no active setlist, <code>computeQueue</code> synthesizes one entry per catalog song (<code>id === songId</code>), so this still resolves.
<code>masterHolderId</code>	<code>string null</code>	which client currently holds the "Master-Token"
<code>masterClaimedAt</code>	<code>number null</code>	

`DEFAULT_SHOW_STATE` : all four fields `null` .

3.5 ShowLog (showLog.ts)

A discriminated union (type) of append-only events. A "show" has no separate summary document — it is simply every event sharing one `showId`. Detection of which show is running, whether a song played long enough to count, and capability transitions all happen client-side, gated to whichever tablet holds the Master-Token (same trust primitive as `ShowState`) — except `note` events, which any device may write.

- **show-started**: { id, showId, type, at: number }
- **song-played**: { id, showId, type, at, endedAt, songId, songTitle } — `songTitle` is denormalized so a report still reads correctly if the song is later renamed/deleted; `endedAt` is simply "when the next song started," since there is no dedicated playback-engine end-of-song signal today.
- **capability-changed**: { id, showId, type, at, capability: string, from: CapabilityStatusValue, to: CapabilityStatusValue }
- **note**: { id, showId, type, at, text: string, authorProfileId: string | null }

`CapabilityStatusValue` = 'available' | 'degraded' | 'missing' (intentionally re-declared here rather than importing the frontend's identical type, since three literal strings aren't worth a cross-package dependency).

3.6 Plugin (plugin.ts)

Mixes plain TypeScript interfaces with two Zod schemas:

- `PluginContext` — { log: { info(msg, meta?), error(msg, meta?) } }.
- `IPlugin` — { name, version, capabilities: CapabilityId[], init(context), shutdown?() }.
- `ShowControlEventSchema` — { type: string (min 1), payload?: Record<string, unknown> }.
- `ShowControlResultSchema` — { status: 'ok' | 'error', message?: string, data?: Record<string, unknown> }.
- `IShowControlPlugin` extends `IPlugin` — adds `trigger(event): Promise<ShowControlResult> | ShowControlResult`. A one-way, fire-and-forget command channel for hardware cues (mixer, lighting, playback engine); real plugins wrap a device SDK, test/dev plugins can be pure in-memory mocks.

3.7 PluginInstallation (pluginInstallation.ts)

A plugin the band has installed; replicated per workspace, so installing on one tablet propagates to the whole mesh (every other tablet + the Stage-Server) via CouchDB. Carries only the plugin's **manifest**, never its code.

Field	Type	Notes
<code>id</code>	<code>string</code> (min 1)	matches a <code>PLUGIN_CATALOG</code> / <code>LOOKUP_CATALOG</code> key on the server

Field	Type	Notes
name	string (min 1)	
version	string (min 1)	
source	string	optional — repository the plugin came from, for a not-yet-built code-loading step
runtime	'client' 'server' 'both'	WebMIDI is a browser API and lives on the tablet; a mixer adapter lives on the Stage-Server — this decides who reports health
capabilities	CapabilityId[]	.default([])
enabled	boolean	.default(true)
installedAt	number (int, ≥0)	

3.8 PluginHealth (pluginHealth.ts)

Runtime reachability of **server-side** plugins, written by the Stage-Server, read by every tablet.

Deliberately separate from `PluginInstallation`: "installed" is a property of the band, "reachable" is a property of tonight's venue.

- `PluginHealthEntry`: { status: 'online' | 'offline' | 'error', lastSeenAt: number (int, ≥0), message?: string }.
- `PluginHealth`: { plugins: Record<string, PluginHealthEntry> } (.default({})).
- `HEALTH_TIMEOUT_MS = 15_000` — nobody ever writes "offline" when the Stage-Server process dies; a stale `lastSeenAt` is the only signal tablets get, so it has to double as one.

3.9 Profile (profile.ts)

One roster entry: { id: string (min 1), name: string (min 1), role: string (min 1) }.

Replicated band-wide (who's in the band is shared knowledge). `role` is a free-text string — bands name their own roles. Which profile a given tablet is currently "acting as" is a **separate, device-local, non-replicated** choice (see `useActiveProfileStore` in §6) — explicitly not authentication; any device can select any profile.

3.10 Dashboard (dashboard.ts)

`Breakpoint = 'sm' | 'md' | 'lg' | 'xl'` (phone / tablet portrait / tablet landscape / stage monitor, per docs/07 §3).

`LayoutItem`: { i: string (min 1), x, y: number (int, ≥0), w, h: number (int, >0), minW?, minH?: number (int, >0) }.

`WidgetInstance`: { i: string (min 1), type: string (min 1), config?: Record<string, unknown> } — `config` is per-**instance**, not per-type, so e.g. a horizontal dashboard-switcher can sit in a top bar while a vertical one sits in a side column on another dashboard.

`Dashboard`:

Field	Type	Notes
<code>id</code>	string (min 1)	
<code>name</code>	string (min 1)	
<code>order</code>	number (int, ≥0)	
<code>widgets</code>	WidgetInstance[]	<code>.default([])</code>
<code>layouts</code>	Partial<Record<Breakpoint, LayoutItem[]>>	<code>.default({})</code> — a breakpoint with no entry inherits <code>react-grid-layout</code> 's own fallback
<code>ownerProfileId</code>	string (min 1)	optional — the "Station" concept: a personal dashboard
<code>ownerRole</code>	string (min 1)	optional — a role-level dashboard; <code>ownerProfileId</code> wins if both are set
<code>visibility</code>	'private' 'public'	<code>.default('public')</code> — enforced client-side only : a private doc still replicates to every tablet (so it survives a device swap), the UI simply doesn't list/select it for anyone but the owner

3.11 Capability (`capability.ts`)

`CapabilityId = z.string().min(1)` — deliberately an open string type, so a future community plugin can bring its own vocabulary. StageBoard's own vocabulary, as a `CAPABILITIES` const:

Key	Value	Meaning
<code>mixer</code>	'mixer'	digital mixer control, IEM "More Me" faders
<code>lighting</code>	'lighting'	DMX/lighting desk cues
<code>showControl</code>	'show-control'	generic MIDI/OSC show-cue sending
<code>midiInput</code>	'midi-input'	foot-switch/MIDI input on the tablet itself (WebMIDI)
<code>audioPlayback</code>	'audio-playback'	backing-track playback engine
<code>backup</code>	'backup'	automated backups

3.12 Lookup (`lookup.ts`)

`LookupResult: { id: string (min 1), title: string (min 1), subtitle?: string, sourceUrl?: string }` — one search candidate, enough to show a picker before committing to a detail fetch.

`ILookupPlugin` extends `IPlugin: { search(query): Promise<LookupResult[]>, fetchDetail(resultId): Promise<Record<string, unknown>> }`. Deliberately **not** `IShowControlPlugin` — that's a one-way fire-and-forget command channel with no room for a query returning multiple candidates; this family is genuinely query/response, routed by a separate `LookupRegistry / LOOKUP_CATALOG` on the server.

4. Backend (packages/core-backend)

A single Fastify process (`src/index.ts`), `logger: true` (structured JSON logging by default).

4.1 HTTP Routes

Method & Path	Purpose	Notes
GET <code>/health</code>	Liveness check	Returns <code>{status:'ok'}</code> unconditionally — does not verify CouchDB reachability or plugin-sync state
GET <code>/songs</code>	Returns a hardcoded in-memory array of one dummy song	Placeholder/stub only — not backed by CouchDB in any way
GET <code>/plugins</code>	Lists currently registered show-control plugins	<code>{name, version, capabilities}[]</code>
POST <code>/plugins/:name/trigger</code>	Fires a show-control event at a named plugin	Body validated against <code>ShowControlEventSchema</code> ; 400 on invalid body, 404 for an unknown plugin name, otherwise returns the plugin's <code>ShowControlResult</code>
GET <code>/lookup/:provider/search?q=</code>	Query-search a lookup plugin (UG scraper or MusicBrainz)	400 if <code>q</code> missing, 404 for unknown provider, 502 + error message on a thrown error (real outbound network I/O can genuinely fail)
GET <code>/lookup/:provider/detail?resultId=</code>	Fetch one search result's full detail	Same error handling as <code>/search</code> . <code>resultId</code> is deliberately a query parameter, not a path parameter: an opaque id can embed a full source URL, and Fastify/ <code>find-my-way</code> 's router rejects any single path segment over 100 characters by default — a real URL clears that easily, while query strings have no equivalent per-segment ceiling. (This was a real, user-reported "Failed to fetch" bug, root-caused and fixed in this project's history — see §10.)

CORS is registered via `@fastify/cors` , `origin` read from `FRONTEND_ORIGIN` env var (comma-split, default `http://localhost:5173`) — for tablets fetching cross-origin from their own dev-server/PWA origin.

Startup sequence: builds a logging adapter bridging plugin `{info,error}` calls onto Fastify's own logger; starts `createPluginSync(...)` (CouchDB connection from `COUCHDB_URL` / `COUCHDB_USER` / `COUCHDB_PASSWORD` env vars, default `http://localhost:5984 / admin / admin` ; workspace from `STAGEBOARD_WORKSPACE` , default

'band-a'); registers `sync.stop()` on Fastify's `onClose` hook; eagerly registers every constructor in `LOOKUP_CATALOG` directly (no CouchDB gating — see §4.5); listens on `PORT` env var (default 3001) on `0.0.0.0`. Any startup error is logged and exits the process.

4.2 CouchDB Client (`couch.ts`)

A minimal, dependency-free HTTP client on Node's global `fetch`, covering exactly the four operations `pluginSync.ts` needs:

- `ensureDb` (PUT, tolerates 412 already-exists)
- `allDocs<T>` (`_all_docs?include_docs=true`)
- `getDoc<T>` (returns `null` on 404)
- `putDoc` (PUT by `_id`, tolerates 409 conflict — "another node wrote first; the next heartbeat re-reads and wins")
- `waitForChange` (long-polls `_changes?feed=longpoll`)

Explicit gap: every request is `Content-Type: application/json`, every body is `JSON.stringify'd` — there is **no support for binary attachments** at all (no multipart PUT, no `_attachments` handling). Audio tracks never touch this client; they replicate directly from the browser's PouchDB straight to CouchDB, bypassing `core-backend` entirely. There is also no bulk-doc (`_bulk_docs`) support and no CouchDB view/query support beyond `_all_docs`.

4.3 Show-Control Plugin Architecture (`registry.ts`, `catalog.ts`)

`PluginRegistry` is a `Map<string, IShowControlPlugin>` wrapper: `register` (calls `init`, stores by name), `list` (projects name/version/capabilities), `unregister` (calls `shutdown` if present, deletes), `trigger(name, event)` (returns `null` for an unregistered name, else delegates).

`PLUGIN_CATALOG`: `Record<string, () => IShowControlPlugin>` mapping `'mock-mixer'` | `'mock-lighting'` | `'mock-playback'` to constructors. A comment marks this hardcoded map as a placeholder for a future GitHub-repo plugin-code-loading mechanism — today, "installing a plugin" means switching on a hardcoded server-side implementation, never loading arbitrary code.

Three mock plugins currently exist, all pure in-memory closures with no real hardware I/O:

- **`mockMixerPlugin`** (`mixer` capability) — simulates a digital mixer (Soundcraft UI24R / Allen & Heath CQ18T stand-in). Holds a `volume` number (starts at 5); `set_volume` updates it if the payload is numeric, any other event type returns the current value unchanged.
- **`mockLightingPlugin`** (`lighting` + `show-control`) — simulates a DMX bridge (QLC+/Maestro stand-in). Tracks `lastCue: string | null`; **every** trigger, regardless of type, sets `lastCue` to the event's `type` and returns it — no filtering of event types at all.
- **`mockPlaybackPlugin`** (`audio-playback`) — simulates a backing-track engine. State `{songId, variantId, isPlaying, positionMs}`; handles `load` / `play` / `pause` / `stop` / `seek` with sensible guards (`play` is a no-op if nothing is loaded; `seek` only applies a numeric payload).

4.4 Plugin Sync (`pluginSync.ts`)

The mechanism turning a band's installed-plugin choices (made in the PWA, replicated to CouchDB) into live plugin instances on the running server. Uses two per-workspace databases: `stageboard-plugins-{workspaceId}` (installation docs) and `stageboard-meta-{workspaceId}` (a fixed-id `plugin-health` heartbeat doc).

- **`readInstallations(docs)`** — parses each doc through `PluginInstallationSchema.safeParse`, silently dropping anything invalid ("a half-replicated doc must not crash the server").
- **`reconcile(installations, registered, catalog)`** — pure, fully unit-tested, no CouchDB dependency. Skips disabled installations and `runtime: 'client'` ones (nothing to start server-side for WebMIDI). For the rest: if the installation's `id` isn't a `PLUGIN_CATALOG` key, it's reported as `unavailable` (installed hardware this server build can't run); otherwise it's `wanted`. Returns `{toRegister, toUnregister, unavailable}`.
- **`createPluginSync(options)`** wires this into a running process: `syncOnce()` reads all installations, reconciles, registers/unregisters accordingly; `writeHeartbeat()` builds a `plugins` map of every currently-registered plugin → `{status: 'online', lastSeenAt: Date.now()}` and writes it (carrying forward the doc's `_rev`); `watchChanges()` is an infinite long-poll loop on the plugins DB (30s timeout) that re-runs `syncOnce()` whenever anything changes, backing off 5s on any error rather than crashing; a `setInterval` fires `writeHeartbeat` roughly every 5s (a third of `HEALTH_TIMEOUT_MS`, comfortably inside the staleness window). `start()` ensures both DBs exist, does one synchronous pass of each, then kicks off the background watch loop.

Net effect: install a plugin in the PWA → doc replicates into CouchDB → server's long-poll wakes → `reconcile` decides to start it → `registry.register` constructs+inits it → next heartbeat reports it "online" to every tablet. Disable/uninstall follows the same path into `toUnregister` → `shutdown()`.

4.5 Lookup Plugin Architecture (`lookupRegistry.ts`, `lookupCatalog.ts`)

Mechanically identical registry pattern to §4.3, but exposing `search / fetchDetail` instead of `trigger`. `LOOKUP_CATALOG` maps `'ultimate-guitar-scraper'` → `createUltimateGuitarPlugin` and `'metadata-lookup'` → `createMusicBrainzPlugin`. Unlike show-control plugins, these are **not** gated behind replicated `PluginInstallation` docs at all — every catalog entry is registered directly at server startup, since they're always-available read-only data sources, not band-installed hardware.

4.6 Ultimate Guitar Scraper Plugin (`ultimateGuitarPlugin.ts`, `ultimateGuitarFormat.ts`)

Uses `puppeteer-core` (not full `puppeteer`, avoiding a bundled Chromium download) to drive a real, already-installed system Chrome/Chromium binary (`resolveChromeExecutable()` : `CHROME_EXECUTABLE_PATH` env var, else a hardcoded candidate list of common install paths). This is a genuine scraper, not a mock: Ultimate Guitar's search page sits behind a Cloudflare bot challenge, so a plain HTTP request only ever returns the JS challenge page — a real browser executing its JS is required, verified live during development. Explicitly scoped as "a private practice tool for personal, non-redistributed use."

A single shared headless `Browser` instance is lazily launched and cached (`--no-sandbox --disable-setuid-sandbox` — a reduced-security posture typical of containerized/root Puppeteer setups); each `search` / `fetchDetail` call opens and closes its own `Page` .

- **`search(query)`** navigates UG's search URL and reads `window.UGAPP.store.page.data.results` (UG's own client-side state, not a public API) via `page.evaluate` . Filters out results with no `type` (paid listings) and explicitly excludes `Pro` / `Official` / `Video` types (no plain importable content). Result `id` is ``${r.id}:${r.tab_url}`` — the full tab URL travels embedded in the opaque id itself, since the plugin is stateless between calls.
- **`fetchDetail(resultId)`** splits on `::` to recover the URL, navigates there, and reads `window.UGAPP.store.page.data.tab_view/.tab` for `content` , `title` , `artist` , `key` (`tonality`), `tuning` , `capo` (genuinely absent, not `0` , when unused — verified against real songs), and `bpm` (from `strummings[0].bpm`). Throws if no content was found. Returns UG's markup converted to ChordPro via `convertUltimateGuitarContent` .

ChordPro conversion (`ultimateGuitarFormat.ts`) — pure, unit-tested logic converting UG's `[ch]Chord[/ch]` tags, `[tab]...[/tab]` wrappers, and `[Section]` headers into ChordPro's inline `[Chord]text / {part: Section}` conventions. The trickiest correctness detail (covered by a dedicated regression test): a chord's column position in the tag-stripped line must account for the chord name's own character width, not just the whitespace gaps between tags — otherwise every chord after the first is misplaced. Standalone chord-only lines with no lyric beneath (intros, back-to-back chord lines, end of song) become inline chords with original spacing preserved rather than being spliced into an unrelated next line. ASCII tab diagrams pass through unchanged apart from losing their wrapper tags.

Known limitations: entirely dependent on UG's undocumented internal `window.UGAPP` state shape (a UG frontend refactor would silently break extraction, with no schema-versioning fallback); requires a real Chrome/Chromium binary manually provisioned on the Stage-Server host (an external, non-npm dependency); no caching (every detail fetch re-scrapes the live page); only free/plain-text tabs are importable, paid/interactive/video-only results are filtered out before the user sees them.

4.7 MusicBrainz Plugin (`musicBrainzPlugin.ts`)

Uses MusicBrainz's official free REST API (not a scraper — the lower-legal-risk choice), sending a descriptive `User-Agent` per their API etiquette (generic/anonymous agents are rate-limited harder). `search(query)` hits `recording/?query=...` and maps results to `LookupResult` (title, artist-first-release-date subtitle, a `musicbrainz.org/recording/<mbid>` source URL). `fetchDetail(mbid)` returns `{title, artist, firstReleaseDate, sourceUrl}` .

Explicit known gap (documented in the plugin's own top comment): the project's vision docs describe wanting BPM/key data from this source, but MusicBrainz only carries identity metadata (title/artist/release date) — no tempo or key. The service that used to provide that, **AcousticBrainz**, **was discontinued**, so this plugin is scoped to confirming song identity only.

4.8 Test Coverage (backend)

Six test files exist under `packages/core-backend/src/plugins/` , currently **43 passing tests**:

- `registry.test.ts` — full `PluginRegistry` lifecycle plus `mockMixerPlugin` behavior (default volume, update, non-numeric payload ignored).
- `lookupRegistry.test.ts` — the parallel lifecycle tests for `LookupRegistry / ILookupPlugin`.
- `mockPlaybackPlugin.test.ts` — full playback state-machine sequence (`load` → `play` → `pause` → `seek` → `stop`) plus guard-clause edge cases.
- `musicBrainzPlugin.test.ts` — mocks global `fetch` ; asserts exact outgoing URL/User-Agent, subtitle-omission logic, empty results, and error propagation on a non-ok HTTP response. No real network calls.
- `pluginSync.test.ts` — covers only the two pure functions `reconcile / readInstallations` (start/stop/ignore/unavailable-catalog scenarios, malformed-doc rejection). The CouchDB-facing side of `pluginSync.ts` itself (`syncOnce` , `writeHeartbeat` , `watchChanges` , `createPluginSync`) has **no test coverage**.
- `ultimateGuitarFormat.test.ts` — thorough coverage of the pure ChordPro conversion logic, including the column-offset regression test described in §4.6.

Untested entirely: `index.ts` (no route-level/integration tests), `couch.ts` (no test at all, mocked or otherwise), `mockLightingPlugin.ts` (no dedicated test file), `ultimateGuitarPlugin.ts` itself (only its pure formatting helper is tested — the Puppeteer-driving code, browser lifecycle, and `resolveChromeExecutable` are not).

5. Frontend Data & Sync Layer (`packages/stage-pwa/src/lib` , `src/store`)

5.1 PouchDB Data Layer

`workspaceDb.ts` provides the shared naming/connection helpers: `localDbName(kind, workspaceId)` (e.g. `stageboard-songs-band-a`), `remoteDbUrl(kind, workspaceId)` (rewrites `VITE_COUCHDB_URL` 's last path segment; returns `null` if unset — how the app runs fully offline), `remoteAuth()`, and `ensureRemoteDbExists()`.

`workspaceCollection.ts` 's `createWorkspaceCollection<T extends {id:string}>(kind)` is the generic pattern every simple collection (songs, variants, setlists, profiles, plugins, show log, dashboards) is built from: `getAll()`, `put(doc)` (re-fetches the existing doc first to carry forward both `_rev` and `_attachments` explicitly — PouchDB only preserves a revision's attachments when the new body explicitly repeats the stub metadata, so without this, saving a song's title after attaching audio would silently delete the attachment), `remove(id)` (no-op if absent), and `startSync(workspaceId)` (opens `db.sync(remoteDb, {live:true, retry:true})` ; deliberately kept a synchronous function, since wrapping the thenable `Sync` object in `await` would unwrap it into its eventual settled result instead of the live handle).

Two singleton documents (`showStateDb.ts` , `pluginHealthDb.ts`) share a per-workspace meta database and follow the identical sync pattern by hand, since they're one fixed-id document each rather than a real collection.

`db.ts` (songs) additionally seeds three dummy songs into an empty workspace, and manages a **legacy** single backing-track attachment (fixed attachment id `'backing-track'`) that predates the variants/multi-track model.

`songVariantsDb.ts`'s **`ensureDefaultVariant(song)`** is the core lazy-migration function: looks for an existing `isDefault` variant for that song; if found, returns it; otherwise builds one (copying `bpm / chordProContent / timecodes` from the `Song` mirror), saves it, and — if a legacy `backing-track` attachment exists — copies it forward as a `band-mix TrackMeta`. Explicitly idempotent and safe to call redundantly from any tablet, since in this local-first mesh there is no single moment "the migration runs" — every tablet calls it lazily on first touch of a legacy song.

5.2 Show-Control / Live Queue Logic

`computeQueue.ts` is a pure function resolving "what's playing now": finds the active setlist (or synthesizes one entry per catalog song if none is active), resolves each entry's variant (`resolveVariantForEntry: explicit variantId → song's isDefault variant → null`), locates the current position via `activeEntryId`, and returns `currentSong / nextSong / currentEntry / nextEntry / currentVariant / nextVariant`. `reorderToPlayNext(entries, entryId, currentEntryId)` implements "play this next" as a pure reordering, without touching `ShowState`.

The Master-Token. `ShowState.masterHolderId` is compared against a per-browser random `clientId` (persisted in plain `localStorage`, independent of Zustand's own `persist`). `claimMaster()` writes the holder unconditionally — how a fresh tablet, or a "Take Over" after a crashed master, acquires write authority.

`setActiveEntry / setActiveSetlist / advanceToNextSong` are all gated client-side on `isMaster` (a UI-level convention, not a CouchDB-enforced permission).

The Master-Clock (`useClockStore`) is a plain, transport-agnostic elapsed-time accumulator: `{isRunning, startedAt, accumulatedMs}` plus `start/stop/reset/seek/getElapsedMs`. It has no notion of "tap" or "audio element" at all — any playback source (manual taps, an `<audio>` element's `timeupdate`, MIDI) drives it purely through this small public API. `useElapsedMs()` is the read-side React hook: subscribes directly to the raw fields (so a `seek()` while stopped renders immediately) and additionally re-renders every animation frame while running.

Show log tracking (`showLogTracking.ts`, `useShowLogTracker.ts`) is master-gated and tracks by **entry id, not songId** (so the same song appearing twice logs as two separate plays). Pure helper rules: a song must stay active ≥ 20 s (`MIN_SONG_DURATION_MS`) to count as "played" (a wrong-song tap corrected quickly doesn't log); a new `showId` starts if the gap since the last activity exceeds 45 minutes (`SHOW_GAP_THRESHOLD_MS`) — long enough to survive a set break, short enough that tomorrow's soundcheck doesn't continue tonight's log. **Documented limitation:** each tablet keeps its own in-memory tracking state; if the Master-Token changes hands mid-song before the 20s window closes, that song is never logged.

`capabilities.ts / useCapabilities.ts` resolves each enabled plugin's declared capabilities into `available / degraded / missing` — client-runtime plugins (WebMIDI) read a live browser probe, server-runtime plugins read the heartbeat doc against `HEALTH_TIMEOUT_MS`; when several plugins provide the same capability, the best status wins. `useCapabilities()` recombines this on every

render via a 5-second ticker (`useNow()`), so a heartbeat going stale greys out a widget on its own, with no explicit event to react to.

5.3 ChordPro Parsing & the Tuner DSP Pipeline

`chordpro.ts` parses ChordPro-with-timecodes (`[01:14.50] I shot the [G] Sheriff`) into structured lines with per-line `partIndex / partLabel` (recognizing both StageBoard's own `{part: X}` directives and standard ChordPro `{soc} / {sov} / {sob}` etc.).

`currentLineIndex / buildPages / nextSectionIndex` are what drive Section Highlighting, the Paginated prompter view, and the MIDI foot-switch's "jump to next section" — all ultimately expressed as a `seek()` against the shared Master-Clock.

The tuner's pitch-detection pipeline is four small, independently tested modules chained together: `pitchDetection.ts` (autocorrelation-based pitch detection with amplitude-relative trimming and parabolic sub-sample refinement), `pitchSmoothing.ts` (a rolling-window **median** — not mean — over recent readings, robust to an occasional octave-detection error, with a miss-counter that clears the window on sustained silence), `noteFromFrequency.ts` (equal-temperament note/cents math, configurable reference pitch for non-440Hz tunings), and `centsColor.ts` (a deliberately two-regime color map — strictly green within ± 2 cents, then a hard jump to orange/red — rather than one smooth gradient that would read as "basically green" for an audibly out-of-tune note).

5.4 MIDI Integration

`webMidi.ts`'s `listenForMidiTriggers` requests `navigator.requestMIDIAccess()` and fires a callback on any Note-On or Program-Change message from any currently- or later-connected input; returns `null` (not an error) when WebMIDI is unsupported or denied. `useMidiTrigger.ts` wires this as one reference-counted subscription shared across every mounted consumer, so multiple UI elements watching foot-switch status don't each double-fire the trigger.

5.5 State Stores

Store	Holds	Populated by
<code>useWorkspaceStore</code>	workspaces, active workspace id	local-only, persisted
<code>useSongsStore</code>	songs	PouchDB <code>songs</code> , live changes feed
<code>useSongVariantsStore</code>	variants (whole catalog)	PouchDB <code>song-variants</code> , live changes feed
<code>useSetlistsStore</code>	setlists	PouchDB <code>setlists</code> , live changes feed; lazily migrates legacy docs
<code>useShowStateStore</code>	show state, <code>clientId</code> , <code>isMaster</code>	singleton PouchDB <code>meta</code> doc, live changes feed
<code>useShowLogStore</code>	events (sorted by <code>at</code>)	PouchDB <code>showlog</code> , live changes feed
<code>usePluginsStore</code>	installed plugins, health	PouchDB <code>plugins</code> + singleton health doc, two live feeds

Store	Holds	Populated by
<code>useProfilesStore</code>	profiles	PouchDB <code>profiles</code> , live changes feed
<code>useDashboardsStore</code>	dashboards, <code>resetNonce</code>	PouchDB <code>dashboards</code> , live changes feed; seeds defaults on first load
<code>useClockStore</code>	elapsed-time accumulator	pure in-memory, no persistence, no PouchDB
<code>useActiveDashboardStore</code>	which dashboard each workspace shows on this device	local-only, persisted, per-device
<code>useActiveProfileStore</code>	which profile each workspace is "acting as" on this device	local-only, persisted, per-device
<code>useEditModeStore</code>	<code>isEditing</code>	local-only, not persisted at all (deliberate)
<code>useThemeStore</code>	theme id, light/dark	local-only, hand-rolled <code>localStorage</code> , applied pre-first-paint
<code>useViewportStore</code>	fullscreen preference	local-only, persisted, per-device

Every workspace-scoped store follows the identical shape: cancel any previous changes-handle, switch the underlying PouchDB to the new workspace's database name, do an initial full read, then attach a live `changes({since: 'now', live: true})` feed that re-populates state on every change — so replicated writes from other tablets or the server propagate automatically, with no polling.

6. Dashboard & Widget System

6.1 The Dashboard System

`App.tsx` is a plain mode-switcher, not a router: `mode==='live'` renders the customizable `Dashboard` ; every other mode (Songs editor, Setlists, Plugins, Backup, Post-show report) is a separate full-screen view reached through the app menu. Every workspace-scoped store is wired through a shared `useWorkspaceResource` hook, so switching the active band re-initializes and re-subscribes every collection at once, dashboards included.

Rendering (`Dashboard.tsx`) uses `react-grid-layout`'s `ResponsiveGridLayout` on a **fixed 12-column × 24-row grid**, with row height/gap computed dynamically from the actually-measured container size so the grid always exactly fills the visible viewport (never a hardcoded guess). The active breakpoint (`sm|md|lg|x1`) is derived from real measured width rather than `react-grid-layout`'s own change-only callback, to avoid a portrait tablet writing edits into the wrong breakpoint's layout.

Notable engineering details:

- **Persistence timing:** layout writes fire only from `onDragStop` / `onResizeStop` , not the generic `onLayoutChange` — avoiding a feedback loop where the widget's own write becomes a CouchDB change, re-renders the `layouts` prop, and RGL reports another spurious "change."

- **Custom "rubber-band" compaction:** a bespoke `Compactor` makes every widget except the one actively being dragged snap back to its pre-interaction position live, frame by frame, the instant it's no longer actually in the way — replacing RGL's default of permanently shoving widgets aside as the dragged one passes over them. A hysteresis ref prevents flicker on a nearly-full grid.
- **Deliberate full remount on dashboard switch** (`key={activeId}:{resetNonce}`) to sidestep an internal RGL bug where its own effect resyncs against stale internal state.
- `normalizeLayout()` only repairs genuinely broken (out-of-bounds/overlapping) layouts; a valid layout with deliberate gaps is returned byte-for-byte unchanged, so a musician's spacing choices survive re-renders.

Edit Mode (`useEditModeStore` , `EditLock.tsx`) is a trivial, deliberately non-persisted boolean, entered via a 600ms long-press (a stray tap on stage must not start rearranging widgets) and exited via a single tap on the "🔒 Fertig" button in `DashboardEditBar` . While editing, the app's floating hamburger menu is hidden entirely, since it would otherwise sit exactly where a widget's resize handle needs to be.

WidgetFrame — the wrapper every placed widget renders inside, and the concrete implementation of Graceful Degradation: computes `isDisabled = status==='degraded'` and, when true, greys the body to 50% opacity, disables pointer events, and overlays an offline badge, while the widget stays in its exact grid position. Its "... menu (portalled to `document.body` , since RGL's own CSS transform would otherwise clip a fixed-position menu to the widget's small box) holds the widget's own config panel plus a visually-isolated "Entfernen" (remove) button.

WidgetLibrary (the "+ Widget" picker) filters the full widget catalog through capability/role gating and free-text search, grouped by category (performance, monitoring, show-control, system-crew, utility, post-show); a widget whose required plugin isn't installed at all doesn't even appear here — distinct from being installed-but-currently-offline, which still appears (greyed).

DashboardManager — full CRUD for the "Station" concept (see §3.10): reorder, rename, duplicate, delete (blocked on the last remaining **public** dashboard, so a fresh device is never left with nothing to show), and create-with-owner (public/shared, or private scoped to a specific profile or an entire role).

Multiple dashboards, per-device state: dashboards themselves replicate band-wide (same mechanism as setlists); *which one is currently shown* is tracked per device in

`useActiveDashboardStore` (persisted locally, never synced) — the singer's tablet and the drummer's can independently sit on different dashboards. Switching between them at runtime is itself a widget (`dashboard-switcher`), so different dashboards can each carry their own navigation placement (horizontal top bar vs. vertical side column).

Default dashboards seeded on first load of a workspace, both using **fixed, hardcoded widget-instance ids** (so two tablets seeding simultaneously converge on the same CouchDB document — an ordinary replication conflict rather than duplicate dashboards):

- **"Prompter"** — a dashboard-switcher top bar, then Next-Song / Clock / MIDI-Status in a row, then the Prompter filling the rest of the 24-row grid.
- **"Monitoring"** — a dashboard-switcher top bar, Clock + MIDI-Status, then IEM "More Me" and Quick Actions side by side, filling the remaining rows.

6.2 Widgets (15 registered types)

Widget	Registry id	Capability required	Summary
Prompter	<code>prompter</code>	— (core)	Shows the current song's ChordPro lyrics/chords, driven off the shared Master-Clock. Two modes: Smooth Scroll (continuously eases the active line to vertical center) and Paginated View (shows one page at a time, flipping automatically on section boundaries).
Live-Queue	<code>live-queue</code>	— (core)	Next 8 upcoming songs from the active setlist. Master-Token holders get a per-row "play next" reorder action; non-masters see a "Master übernehmen" prompt instead.
Next Song	<code>next-song</code>	— (core)	Compact current/next song + BPM strip; its advance button is disabled unless this device holds the Master-Token.
Show Cockpit / Clock	<code>clock</code>	— (core)	Start/Stop/Reset for the shared Master-Clock, mm:ss display.
Fußtaster / MIDI-Status	<code>midi-status</code>	<code>midi-input</code>	Live WebMIDI connection status (unsupported / no device / connected) plus a manual "simulate foot switch" button.
Dashboard-Umschalter	<code>dashboard-switcher</code>	— (core)	Switches which dashboard is active on this device; configurable horizontal/vertical, and can be pinned to specific target dashboards.
More Me / IEM	<code>iem-more-me</code>	<code>mixer</code>	Three vertical faders (vocals/guitar/band). Local-state only today — no low-latency channel to a real mixer exists yet, so moving a fader has no hardware effect regardless of plugin status.
Quick Actions	<code>quick-actions</code>	<code>show-control</code>	A grid of big cue buttons (Strobo/Blackout/Kaltfunken/Talkback). UI stub — taps only set local highlight state, pending a low-latency cue channel.
Lighting Cues	<code>lighting-cues</code>	<code>lighting</code>	Same underlying grid component as Quick Actions, different cue labels (Voll/Dimmen/Chase/Farbwechsel). Also a UI stub.
Show-Playback	<code>show-playback</code>	<code>audio-playback</code>	Fully wired end-to-end: Play/Pause/Stop send real <code>ShowControlEvents</code> over HTTP to the Stage-Server via the plugin providing <code>audio-playback</code> , and the widget reflects the plugin's actual reported state (position, <code>isPlaying</code>).
System-Status	<code>system-health</code>	— (core, <code>relevantRoles: ['Crew']</code>)	Traffic-light overview of every capability the band's plugins expose — the soundcheck/setup cockpit view, only offered to a "Crew" profile role.

Widget	Registry id	Capability required	Summary
Stimmgerät / Tuner	tuner	— (uses <code>getUserMedia</code> directly, not the plugin system)	Live pitch detection via the Web Audio API; manages its own degradation states (denied / insecure-context / unsupported) since it needs a browser API, not a plugin. Fully configurable (sensitivity, responsiveness, note naming, reference pitch 400–480Hz).
Backing-Track Player	backing-track-player	— (core)	Home-practice playback: independently picks its own song (not tied to the live queue), plays a variant's audio track, and mirrors playback position into the shared Master-Clock so the Prompter tracks real audio.
Show-Notizen	show-notes	— (core)	Free-text notes into the show log, attributed to the active profile. Deliberately not Master-Token-gated — any musician or crew member can log an ad-hoc note.
Backup-Status	backup-status	backup	Minimal glanceable status dot; the full backup management UI lives in the separate "Backup" app mode.

Cross-cutting pattern: a single shared Master-Clock drives every time-based widget, and a single Master-Token concept gates who may advance the live queue — both surfaced consistently as a "Master übernehmen" fallback wherever relevant. Real hardware integration is currently a two-tier story: Show-Playback is genuinely wired to the backend; IEM faders and both cue-grid widgets are deliberate UI-only stubs pending a lower-latency cue channel (see §10).

7. Views & Feature Components

7.1 Navigation (`AppMenu.tsx`)

The app's central menu (opened via a floating hamburger button, hidden while editing the dashboard) groups: **Ansicht** (mode switcher: Live/Songs/Setlists/Plugins/Backup/Nachbericht — Backup is hidden entirely unless a backup plugin is installed), **Dashboard** (the edit-mode lock, live-mode only), **Band** (`WorkspaceSwitcher`), **Profil** (`ProfileSwitcher`), **Darstellung** (`ThemeSwitcher`), and **Anzeige** (a fullscreen toggle, only shown if supported).

7.2 SheetEditor — Song/Variant Editor

Reached via "Ansicht" → "Songs". Two-pane layout: an editing form on the left, a live `ChordProLyrics` preview on the right.

- A song dropdown selects/creates a song; "+ Neuer Song" starts a blank draft. Selecting a song runs `ensureDefaultVariant` (migrating legacy data on the fly).
- A variant dropdown (hidden while editing a new/unsaved song) lists all of the song's variants; "+ Neue Variante" starts a second arrangement from a blank chord sheet while keeping the title/artist.

- Title/Band/BPM/Key/Tuning/Capo inputs edit the current draft (Capo clears to genuinely `undefined`, not `0`, when emptied).
- `TrackManagerField` is embedded, disabled until the draft is first saved (there's no document id to attach audio to yet).
- The ChordPro text area has buttons to insert `{part: Verse|Chorus|Bridge|Solo}` directives at the caret, a "Song importieren" button (opens `TabImportOverlay`), and a "Tap-to-Sync starten" button that swaps the text area for `TapToSync`.
- **Save** validates both the `Song` and `SongVariant` documents with their Zod schemas before writing; it rebuilds the `Song`-level mirror fields from whichever variant is actually `isDefault` — never from whichever variant happens to be open — so editing a non-default arrangement can't corrupt what un-migrated widgets still read.
- **Import merge**: only overwrites `artist/key/tuning/capo/bpm` when the imported source actually provided a value, so a tab missing e.g. capo data never silently erases a capo already set in the editor.

7.3 TabImportOverlay — Ultimate Guitar / MusicBrainz Import

A modal opened from SheetEditor. Provider picker (UG scraper or MusicBrainz) → search → result list → detail preview → import. Each result with a source URL gets an "↗" button opening a **real popup window** of the actual source page — a deliberate substitute for an `<iframe>`, since Ultimate Guitar (like most sites) sends `Content-Security-Policy: frame-ancestors 'self'`, which no amount of app-side code can override; the popup is a genuine, live, scrollable copy of the real page, just not embedded inline. The detail pane shows a metadata summary (Band/Key/Tuning/Capo/BPM, only populated fields) above the raw ChordPro preview for an importable result, or a generic key/value dump with an explanatory note for a MusicBrainz result (which has no chord/lyric data to import, only identity confirmation).

7.4 TapToSync — Audio-Driven Recording Mode

Replaces the ChordPro text area while active. Splits the song into tappable lines (skipping blanks and `{part: ...}` directives). If the variant has an audio track (band-mix preferred, else reference), a real `<audio>` element with Play/Pause drives the shared Master-Clock via its native playback events — tapped timestamps land on the actual recording's position rather than a hand-started stopwatch that would drift out of sync. Without a track, the clock starts immediately as a bare stopwatch (the original, pre-audio-integration behavior, still fully supported). Space bar or the "Tap" button stamps the current line and advances to the next tappable one; reaching the end calls back with the fully re-timestamped content. "Abbrechen" discards everything without touching the saved song.

7.5 ChordProLyrics — Shared Renderer

Used by both SheetEditor's preview and the Prompter widget. Renders parsed ChordPro lines with chords positioned above their lyric text, section-part headers, optional active-line highlighting, and pagination support (a `startIndex` offset so a paginated caller can render one slice while indices stay absolute across the whole song). A specific spacing fix ensures a part label's first line's chord never

visually collides with the label text above it (a real bug found and fixed during this project's history — see §10).

7.6 TrackManagerField — Multi-Track Audio Manager

Embedded in SheetEditor, scoped to the open variant. Lists attached tracks (Referenz/Band-Playback/Stem) each with a remove button; a track-kind selector plus a file upload control attaches a new audio file as a PouchDB attachment on the variant document.

7.7 SetlistManager — Setlist CRUD & Per-Entry Variant Selection

Reached via "Ansicht" → "Setlists". Left pane lists all setlists (with an active marker, Duplicate — regenerating fresh entry ids so edits to the copy never affect the original — and Activate, gated to the Master-Token holder). Right pane shows the selected setlist's entries, each with a variant dropdown (only shown if the song actually has more than one variant), reorder arrows, and remove. "Song hinzufügen" explicitly allows adding a song already present in the setlist — the mechanism supporting a song appearing twice under different arrangements.

7.8 PluginManager — Install/Configure Show-Control Plugins

Reached via "Ansicht" → "Plugins". A hardcoded catalog of 5 mock plugins (Mixer, WebMIDI Input, Lighting/DMX, Backup, Playback) can be installed (writing a replicated `PluginInstallation` doc — distributing the choice to the whole stage mesh) or removed; installed plugins show a health label derived from the server heartbeat (client-runtime plugins always show "läuft auf dem Tablet", needing no heartbeat) and an enable/disable toggle.

7.9 BackupManager — Backup Plugin Status View

Reached via "Ansicht" → "Backup" (hidden from the menu entirely unless a backup-capable plugin is installed). Read-only status view: the header text is explicit that StageBoard itself never triggers a backup — that's a plugin's job — this view only shows whether one is installed and reachable; actual install/configure happens in the separate Plugin Manager.

7.10 PostShowReport — "Nachbericht"

Reached via "Ansicht" → "Nachbericht". Groups the append-only show log by `showId` (a "show" has no separate document — it's simply the events sharing an id), sorted newest-first, rendering each event type distinctly: song plays (title, start time, duration), capability transitions (amber warning line), and free-text notes (with author, resolved from the profile roster).

7.11 Profile / Theme / Workspace Switchers

- **ProfileSwitcher** — picks which roster profile a device is currently "acting as." Explicitly not authentication (any device can pick any profile); controls only which private dashboards are visible. Supports create/edit/delete of profiles, which are themselves band-replicated.
- **ThemeSwitcher** — picks a visual theme; a light/dark toggle only appears for the one theme that supports both (the other four are dedicated dark stage looks). Per-device, not replicated.

- **WorkspaceSwitcher** — picks/creates a band ("workspace"). Deliberately tucked into the menu rather than a floating control, since band-switching happens rarely (session start).
-

8. Infrastructure & Developer Tooling

- **docker-compose.yml** — single `couchdb:3` service, port 5984, dev-only `admin / admin` credentials, persisted named volume.
 - **scripts/setup-couchdb.sh** — idempotent bootstrap: waits for CouchDB to answer, finishes single-node cluster setup, creates the app database, enables browser-facing CORS. No per-tablet user accounts are created.
 - **scripts/tablet-debug.mjs** — a live tablet-debugging workflow over Chrome DevTools Protocol via `adb forward`, with `list / eval / watch` subcommands, letting a tablet-only bug (touch-drag timing, real screen geometry) be inspected and console-watched live without a manual DevTools window or screen-share.
 - **Node version**: pinned to 24 via `.nvmrc / nvm` (not the system `apt` Node 18, unpatched since April 2025). A documented `npm_config_prefix` gotcha can silently stop `nvm` auto-activating the pinned version in some shells.
 - **Structured logging** is a stated requirement (docs/03): every log entry should carry timestamp/level/source/correlation-id; the current codebase satisfies this on the backend via Fastify's built-in Pino logger, but a dedicated frontend "Live-Debug-Console" (Home-Assistant-style) is not yet built — the interim workaround is a per-feature `localStorage` debug-flag pattern (`lib/gridDebug.ts`) combined with the CDP tablet-debug script above.
 - **Testing philosophy**: strict hexagonal architecture — the plugin interfaces (`IShowControlPlugin` , `ILookupPlugin`) exist specifically so every real hardware integration ships with a pure, hardware-free mock counterpart, allowing both automated tests and manual verification without any physical gear connected.
-

9. Testing & Quality Summary

As of this commit, the automated test suites pass in full:

- **packages/core-backend** : 6 test files, **43 tests**, all passing (`vitest run`). See §4.8 for coverage detail and gaps.
- **packages/stage-pwa** : 13 test files, **182 tests**, all passing (`vitest run`). Covered: `capabilities` , `centsColor` , `chordpro` , `computeQueue` , `dashboardLayout` , `id` , `modes` , `noteFromFrequency` , `pitchDetection` , `pitchSmoothing` , `showLogTracking` , `useClockStore` , `tunerConfig` .
- **Not covered by any automated test**: all React components/widgets (no component-level or rendering tests exist anywhere in the frontend — verification of UI behavior has so far been manual, browser-driven verification during development), `couch.ts` , `index.ts` (backend

routes), `mockLightingPlugin.ts`, and the Puppeteer-driving parts of `ultimateGuitarPlugin.ts`.

- Both packages are typechecked in strict TypeScript (`tsc --noEmit`) and linted (ESLint, with `eslint-plugin-react-hooks` for the frontend) as part of the normal development workflow; both currently pass with zero errors/warnings.

10. Known Limitations & Gaps (current state)

This section lists concrete, verified gaps in the current implementation — not speculative future work (see §11 for that).

Backend / infrastructure

- `GET /songs` on the Stage-Server returns a hardcoded dummy array — it is not connected to CouchDB and is effectively dead code left over from early development.
- `couch.ts` has no binary-attachment support; the Stage-Server cannot itself read or write audio track data — all track attachments flow directly from browser PouchDB to CouchDB.
- No per-tablet/per-band CouchDB user accounts exist; every client and the server authenticate as one shared admin user (`admin / admin` by default) — acceptable for a private LAN dev setup, not yet a real multi-tenant security boundary.
- The Ultimate Guitar scraper depends entirely on UG's undocumented internal client-state shape and requires a real Chrome/Chromium binary to be manually installed on the Stage-Server host; it runs with `--no-sandbox`.
- MusicBrainz's lookup plugin cannot provide BPM or musical key — that data source (AcousticBrainz) no longer exists.
- No automated route-level/integration tests exist for the Fastify server itself, nor for the CouchDB client.
- Only a hardcoded map of 3 mock show-control plugins exists; real hardware adapters (actual mixer/lighting protocols) and the "load plugin code from a GitHub repo" mechanism described in the vision docs are not built — installing a plugin today only ever switches on one of these mocks.

Frontend

- The IEM "More Me" widget and both cue-grid widgets (Quick Actions, Lighting Cues) are UI-only stubs: they hold local state and visually respond to taps, but there is no low-latency channel (OSC/WebSocket) to actually drive a mixer or lighting desk yet. Only the Show-Playback widget is wired end-to-end to the backend.
- `useShowLogTracker`'s song-play detection is per-tablet, in-memory state; a Master-Token handoff mid-song (before the 20-second confirmation window closes) causes that play to go unlogged.
- There is no explicit "Pause/Stop" show-control state — `ShowState` only ever knows which song is active, so the post-show report cannot distinguish an intentional pause (banter, retuning) from the previous song's actual play duration; both collapse into the same timestamp.

- The fully personal, per-device "Station" layout vision from docs/07 is only partially realized: the shared/replicated Dashboard concept (with private/role-scoped visibility) is implemented; a dedicated best-of-breed automatic per-device layout beyond manually building/choosing a Dashboard is not.
- No component-level or UI rendering tests exist anywhere in the frontend; all UI verification to date has been manual/browser-driven during development.

11. Roadmap — Planned but Not Yet Implemented

The following phases were scoped in the project's internal implementation roadmap and are **not yet built** in any form (no schema, no UI, no backend support exists for them today):

1. **Async-Job Infrastructure + YouTube Reference-Track Extraction.** A generic replicated `AsyncJob` document type (`queued/running/done/error` , `progress`) that the Stage-Server would watch and update via the same long-polling pattern already used for plugin sync; concretely, first used to shell out to `yt-dlp` to extract a YouTube recording as a `reference` track. Flagged in the roadmap itself as a YouTube-ToS gray area, accepted only as a private practice tool.
2. **Cue Schema + Manual Cue Recording.** A `ShowCue` document type (timestamped MIDI/OSC/lighting/note payloads, variant-scoped) plus a `CueRecorder` UI (structurally similar to the now-built `TapToSync`) for a technician to record device cues against a real track's playback, capturing raw WebMIDI messages with timestamps.
3. **Automatic Cue-Detection Assist.** A zero-ML first pass suggesting candidate cue points from ChordPro's own part-boundary data (already computed by `chordpro.ts` today) plus simple Web-Audio-API onset/transient detection — explicitly scoped as a pragmatic starting point, with deeper audio-ML detection flagged as open research, not committed.
4. **Live Cue Firing + Post-Show Persistence.** Routing recorded/suggested cues through the existing Show-Control Gateway during a live show (Master-Token gated, same as song advancement), logging fired cues into the append-only show log, and extending `PostShowReport` to offer saving a night's fired cues back into the variant's cue list for next time.
5. **Stem Separation.** An explicit build-vs-buy spike between client-side WebGPU-based separation (matching the vision docs' stated preference, but needing graceful degradation for weaker devices) and a server-side queue (e.g. Demucs) reusing the async-job infrastructure from item 1 — deferred pending that evaluation. The `SongVariant / TrackMeta` schema already supports a `kind: 'stem'` track with `parentTrackId` today; only the automated separation pipeline itself is missing.

None of the above should be inferred as "in progress" — they are scoped and ordered in the roadmap but have not been started.

12. Appendix: Package Manifest

Package	Version	Key dependencies
shared-types	0.0.1	zod ^4.4.3
core-backend	0.0.1	fastify ^5.12.1, @fastify/cors ^11.3.0, puppeteer-core ^25.9.0, shared-types
stage-pwa	0.0.0	react ^18.3.1, pouchdb-browser ^9.0.0, zustand ^5.0.15, react-grid-layout ^2.2.4, shared-types; dev: vite ^8.2.2, tailwindcss ^3.4.19, vitest ^4.1.11, typescript-eslint ^8.11.0

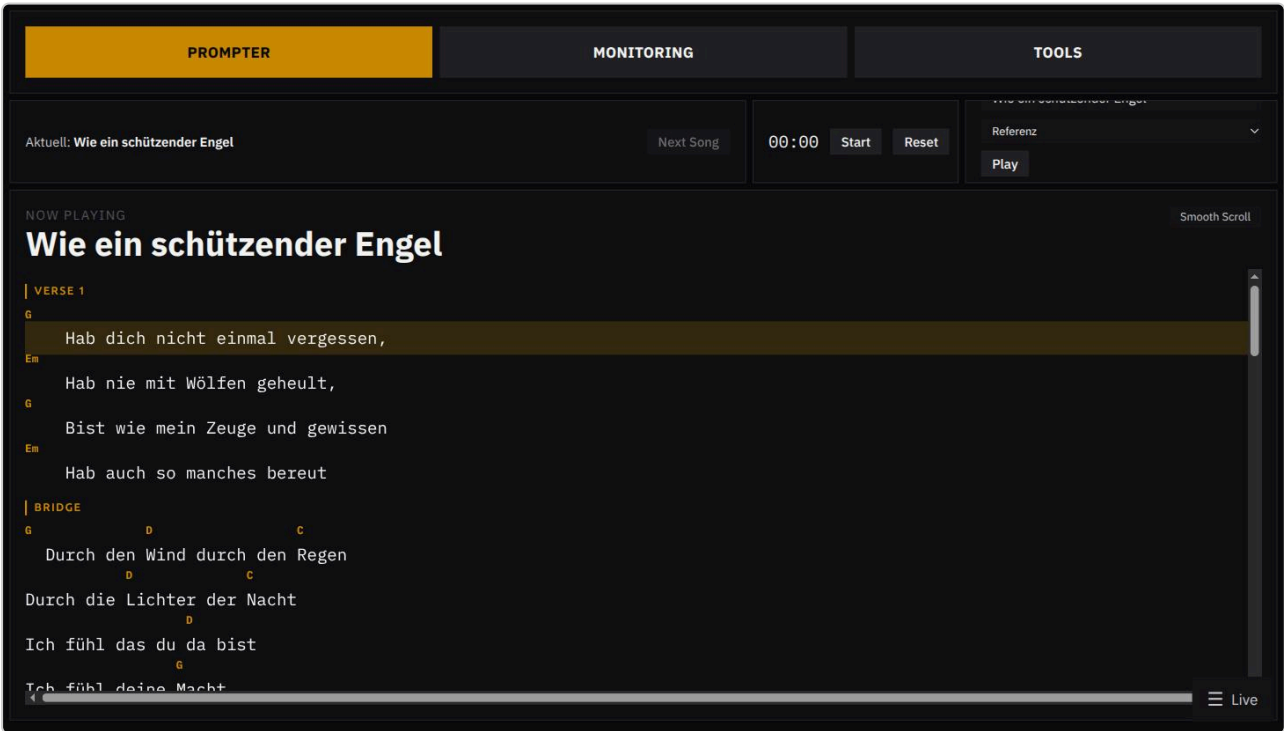
Root workspace script: `npm run dev` runs both `core-backend` and `stage-pwa` dev servers concurrently (labeled, color-coded output via `concurrently`).

13. Screenshot Gallery

Live captures from the running application (dev environment, commit `3d39837`), one per major view/popup/submenu. Several intentionally show real degraded/offline states and real data from development testing, rather than an artificially clean demo — this is what the app actually looks like today.

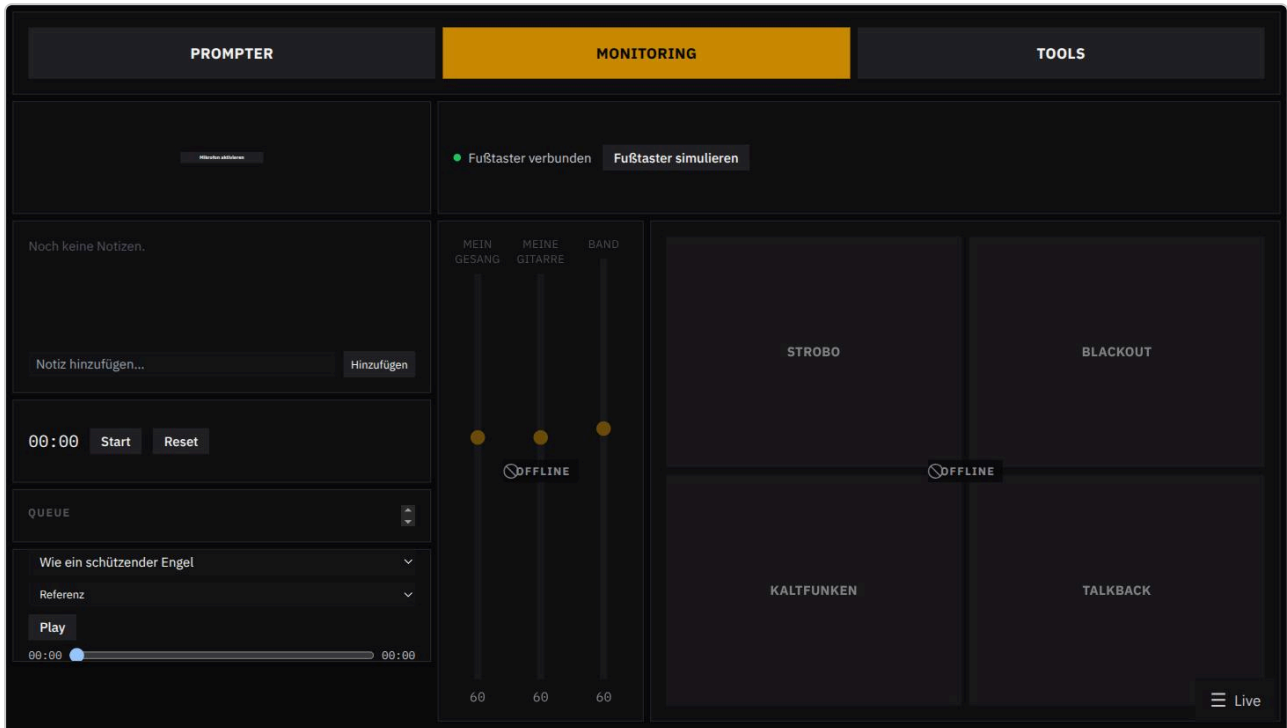
Live — Prompter dashboard

The default live view: current song's ChordPro lyrics with chords, Master-Clock controls, next-song button, and the dashboard-switcher tab bar (Prompter/Monitoring/Tools).



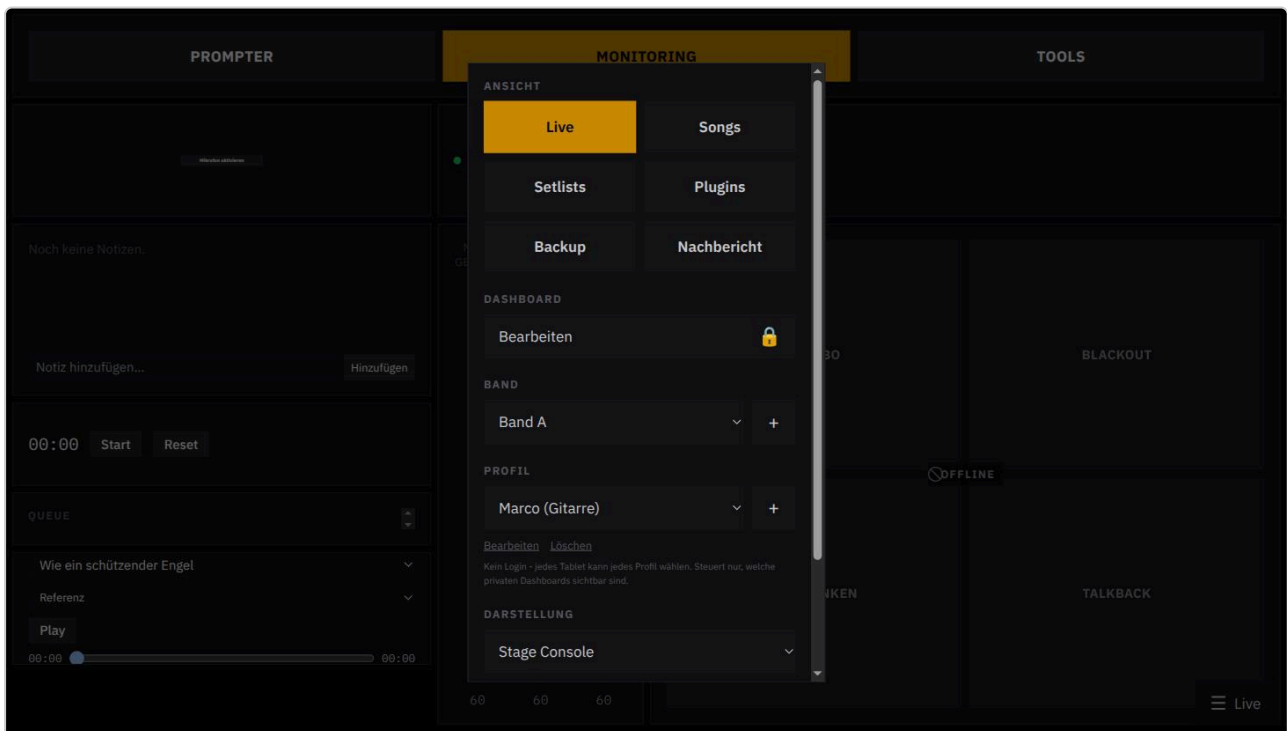
Live — Monitoring dashboard (Graceful Degradation)

The Monitoring dashboard, showing the Tuner, MIDI foot-switch status (connected), IEM "More Me" faders, and the Quick Actions cue grid. Both the IEM and Quick Actions widgets show the "OFFLINE" degraded overlay from `WidgetFrame`, since no plugin providing `mixer / show-control` was active for this workspace at capture time — the widgets stay in place, greyed out, exactly as Graceful Degradation is supposed to behave.



App Menu

The central navigation menu: Ansicht (mode switcher), Dashboard edit-lock, Band/Profil/Darstellung switchers.



Songs — SheetEditor

The song/variant editor: song and variant selectors, title/band/BPM/key/tuning/capo fields, track manager, and a live ChordPro preview on the right. Tap-to-Sync-recorded timecodes are visible in the ChordPro text.

The screenshot displays the SheetEditor interface. On the left, the 'Song' section shows 'Wie ein schützender Engel' with a '+ Neuer Song' button. Below it, the 'Variante' section shows 'Original' with a '+ Neue Variante' button. The 'Titel' and 'Band' fields are both 'Wie ein schützender Engel'. The 'BPM' is set to 120, 'Key' is empty, 'Tuning' is empty, and 'Capo' is 3. The 'Tracks' section shows 'Referenz: Referenz' and 'Band-Playback: Band-Playback' with 'Entfernen' buttons. The 'ChordPro-Text' section has buttons for '+ VERSE', '+ CHORUS', '+ BRIDGE', and '+ SOLO', and a 'Song importieren' button. The 'ChordPro-Text' area shows a list of chords and lyrics for 'VERSE 1' and 'BRIDGE'. On the right, the 'ChordPro preview' shows the lyrics and chords for 'VERSE 1', 'BRIDGE', and 'REFRAIN'. The 'Songs' button is visible in the bottom right corner.

Song Importieren — Ultimate Guitar result (live scrape)

A real, live search against Ultimate Guitar for "wonderwall" through the Puppeteer-driven scraper plugin, with the detail pane showing extracted metadata (Band, Key, Tuning, BPM) and converted ChordPro content — not a mock.

The screenshot displays the SheetEditor interface with the 'Song Importieren' dialog box open. The dialog box has a search bar with 'wonderwall' and a 'Suchen' button. Below the search bar, there is a list of search results for 'Wonderwall' by Oasis. The selected result shows the following metadata: Band: Oasis, Key: F#m, Tuning: E A D G B E, BPM: 87. The 'ChordPro-Text' section shows the lyrics and chords for 'VERSE 1' and 'VERSE 2'. The 'Importieren' button is visible at the bottom right of the dialog box.

Tap-to-Sync (audio-driven)

Recording mode with a real attached track: Play/Pause and a position/duration readout drive the Master-Clock, and tapped lines show their captured timecodes.

The screenshot displays the Tap-to-Sync interface. On the left, the song details for "Wie ein schützender Engel" are shown, including the title, BPM (120), Key, Tuning, and Capo (3). The "Tracks" section shows the current track and its duration (00:00 / 00:00). Below this, the lyrics are displayed with timecodes: [00:09.16] [G] Hab dich nicht einmal vergessen, [00:13.38] [Em] Hab nie mit Wölfen geheult, [00:17.43] [G] Bist wie mein Zeuge und gewissen [00:21.70] [Em] Hab auch so manches bereut. The right panel shows the lyrics with guitar chords (G, Em, D, C) and a "Songs" button at the bottom right.

Setlists — SetlistManager

A real setlist ("Maihingen") with its ordered entries; entries whose song has more than one variant show a variant picker inline (e.g. "Original").

The screenshot displays the SetlistManager interface. On the left, the "SETLISTS" section shows a list of setlists: "Maihingen (4) • aktiv", "Test (6)", "Test Set (4)", and "tablet (2)". Each setlist has "Duplizieren" and "Aktivieren" buttons. Below this, there is a "Setlist deaktivieren (alle Songs)" button. On the right, the "MAIHINGEN" setlist is shown with four entries: "1. Highway to Hell", "2. Mustang Sally", "3. Sweet Caroline", and "4. Wie ein schützender Engel". Each entry has a variant picker (e.g., "Original") and up/down arrows. Below the setlist, there is a "Song hinzufügen" button and a "Song wählen..." dropdown menu. A "Setlists" button is at the bottom right.

Plugins — PluginManager

All five catalog plugins installed, showing real health states: WebMIDI running client-side needs no heartbeat; Mock Backup shows "kein Heartbeat" because it isn't in the server's `PLUGIN_CATALOG` at all (a genuine, permanent "unavailable" case, not a transient one); the rest show "Heartbeat veraltet" shortly after a server restart.

Plugins

Installierte Plugins replizieren über das Bühnen-Netz zu allen Tablets und zum Stage-Server. Deaktivierte Plugins verschwinden aus der Widget-Bibliothek.

Generic WebMIDI Input v0.0.1 midi-input · läuft auf dem Tablet	Aktiv	Entfernen
Mock Backup v0.0.1 backup · kein Heartbeat – Stage-Server offline?	Aktiv	Entfernen
Mock Lighting (DMX) v0.0.1 lighting, show-control · Heartbeat veraltet	Aktiv	Entfernen
Mock Mixer v0.0.1 mixer · Heartbeat veraltet	Aktiv	Entfernen
Mock Playback v0.0.1 audio-playback · Heartbeat veraltet	Aktiv	Entfernen

VERFÜGBAR

Alles aus dem Katalog ist installiert.

≡ Plugins

Backup — BackupManager

The read-only backup status view, showing the same Mock Backup plugin and its permanently-unreachable state, with the explanatory text that StageBoard itself never triggers backups.

Backup

Automatische Backups laufen über ein installiertes Backup-Plugin, nicht über StageBoard selbst - hier siehst du nur, ob eins installiert und gerade erreichbar ist. Ein Plugin verwalten geht unter „Plugins“.

Mock Backup v0.0.1 kein Heartbeat – Stage-Server offline?	Aktiv
---	-------

≡ Backup

Nachbericht — PostShowReport

Real capability-transition events captured live during today's development/testing session, grouped by show.

Nachbericht

Automatisch erfasst: wann eine Show begann, welche Songs wirklich gespielt wurden (mindestens 20 Sekunden aktiv), technische Ereignisse und Notizen von Band und Crew.

26.08.2026, 23:58

26.08.2026, 23:57

- ⚠ lighting: degraded → available (26.08.2026, 23:57)
- ⚠ mixer: degraded → available (26.08.2026, 23:57)
- ⚠ show-control: degraded → available (26.08.2026, 23:57)
- ⚠ audio-playback: degraded → available (26.08.2026, 23:57)
- ⚠ lighting: available → degraded (26.08.2026, 23:58)
- ⚠ show-control: available → degraded (26.08.2026, 23:58)
- ⚠ mixer: available → degraded (26.08.2026, 23:58)
- ⚠ audio-playback: available → degraded (26.08.2026, 23:58)

26.08.2026, 23:57

- ⚠ audio-playback: degraded → available (26.08.2026, 23:57)
- ⚠ show-control: degraded → available (26.08.2026, 23:57)
- ⚠ mixer: degraded → available (26.08.2026, 23:57)
- ⚠ lighting: degraded → available (26.08.2026, 23:57)

26.08.2026, 23:33

- ⚠ audio-playback: degraded → available (26.08.2026, 23:33)
- ⚠ show-control: degraded → available (26.08.2026, 23:33)
- ⚠ mixer: degraded → available (26.08.2026, 23:33)

Nachbericht

Dashboard Edit Mode

Edit Mode unlocked: the `DashboardEditBar` toolbar (+ Widget / Dashboards verwalten / Zurücksetzen / Fertig) and every widget showing its `react-grid-layout` drag/resize handles and label.

EDIT Monitoring + Widget Dashboards verwalten Zurücksetzen Fertig

Dashboard-Umschalter

PROMPTER

Stimmgerät

Stimmgerät

Noch keine Notizen, Show-Notizen

Notiz hinzufügen...

Hinzufügen

Show Cockpit

Start

Reset

QUEUE

Live-Queue

Wie ein schützender Engel

Backing-Track Player

Play

MONITORING

Fußtaster

Fußtaster verbunden

Fußtaster simulieren

MEIN More Me (IEM)

MEINE ARRE

BANK

OFFLINE

OFFLINE

OFFLINE

TOOLS

Quick Actions

STROBO

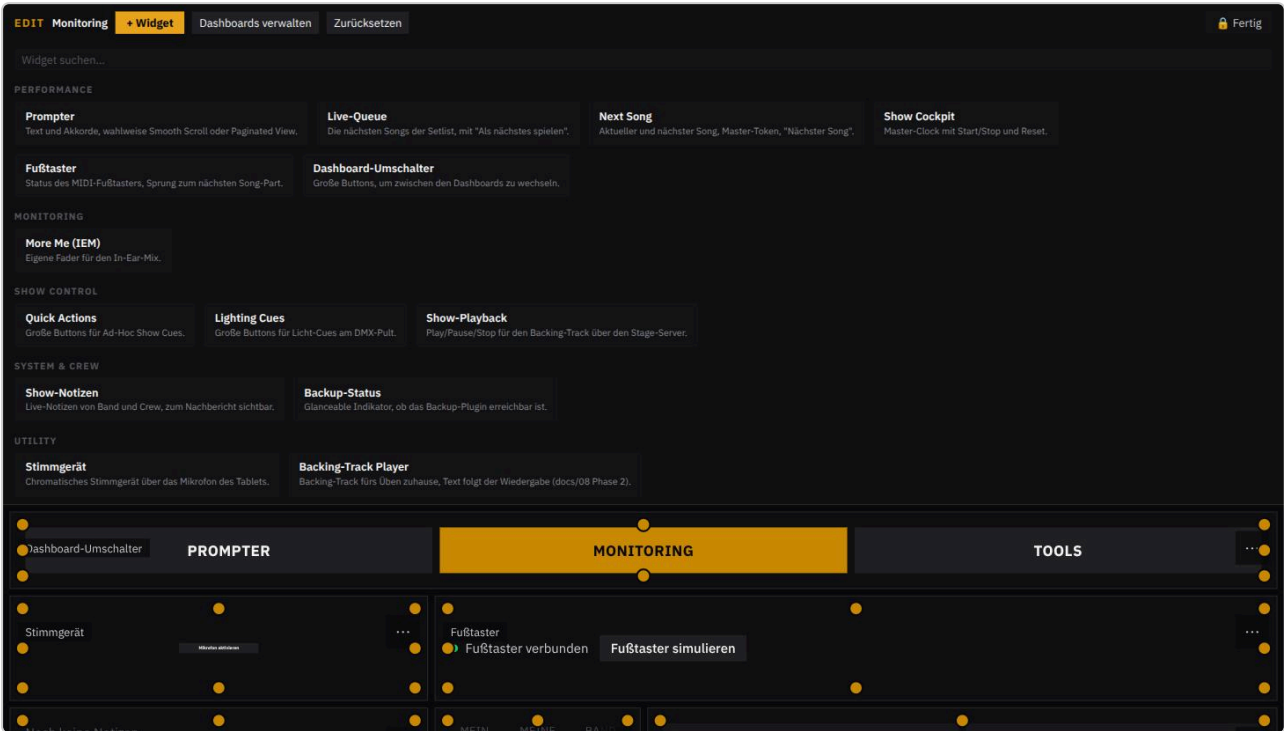
BLACKOUT

KALTFUNKEN

TALKBACK

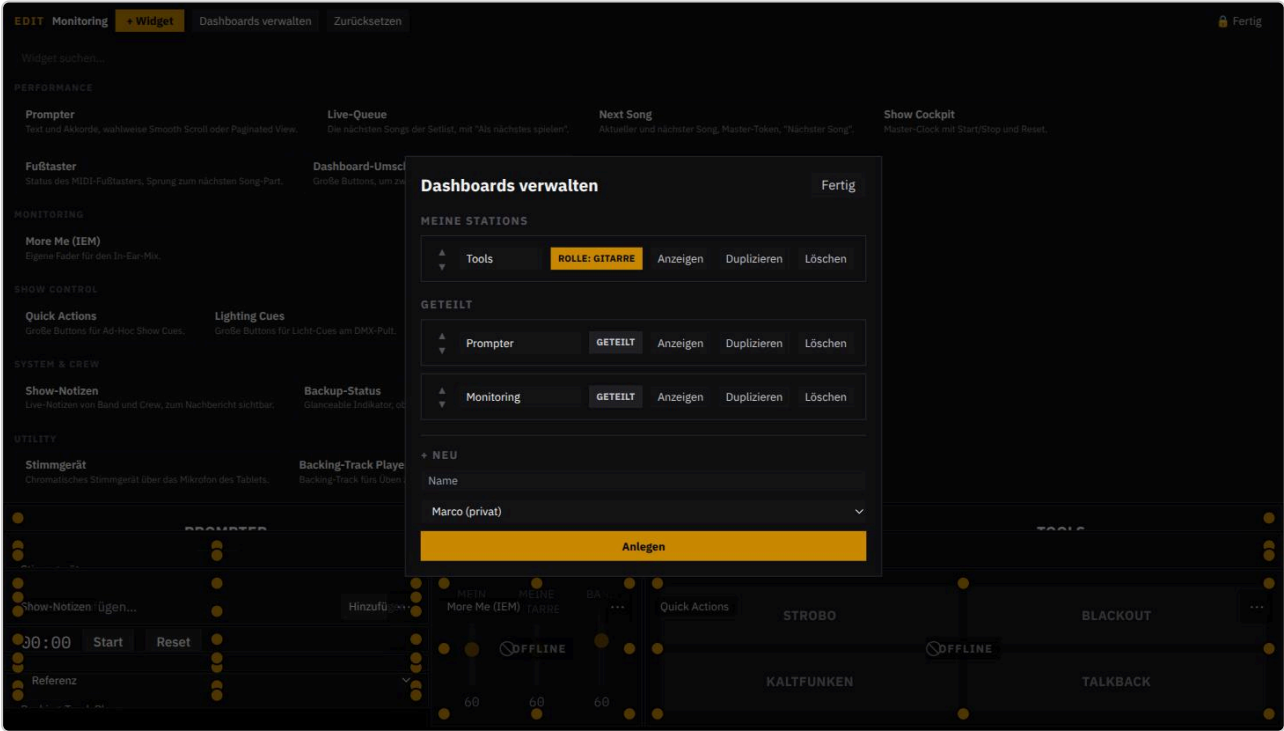
Widget Library

The "+ Widget" picker, grouped by category. Note "System-Status" does not appear here at all — it's gated to a "Crew" profile role, and the active profile at capture time was "Marco (Gitarre)."



Dashboard Manager ("Stations")

Full CRUD over dashboards, showing a real personal "Station" ("Tools", scoped to the "Gitarre" role) alongside the two shared default dashboards.



Widget Config Panel (Tuner)

The Tuner widget's own configuration popover: sensitivity, responsiveness, reference pitch, and note-naming convention, plus the "Entfernen" (remove) action every widget menu carries.



Generated from a full source-level audit of the repository at commit `3d39837` . This document describes what the code actually does; where the project's design docs (`docs/01` – `docs/08`) describe a broader future vision, that distinction is called out explicitly above rather than blurred.